



Argo CD

Zero to Production

GITOPS

DEPLOY

MANAGE

OPERATE



**Git
Repository**



Argo CD



**Development
Cluster**



**Staging
Cluster**



**Production
Cluster**



**PRACTICAL
EXAMPLES**



**REAL-WORLD
DEPLOYMENTS**



**TROUBLESHOOTING
GUIDES**



**PRODUCTION
BEST PRACTICES**



**Instagram
@veriqta**



**GitHub
@veriqta**



**LinkedIn
@veriqta**



**Telegram
@veriqta**



**Medium
@veriqta**



**X
@veriqta**

Argo CD: What It Is and Why It Exists

GitOps, desired state, continuous delivery, where Argo CD fits

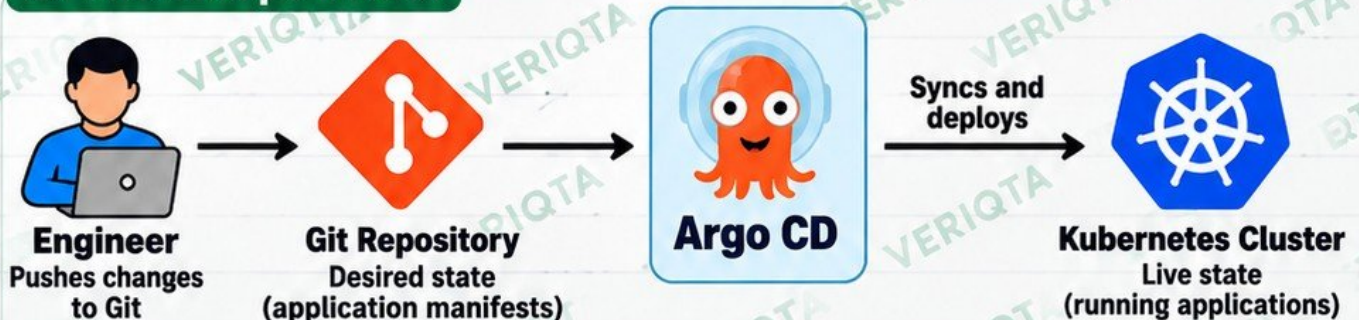
1. What is Argo CD?

- **Argo CD** is a Kubernetes Native continuous delivery tool.
- It automatically deploys and keeps your applications in sync with a **Git repository**.
- It follows the **GitOps** model.

2. Why Does It Exist?

- Traditional deployments are manual, inconsistent and error prone.
- Argo CD exists to automate deployments using **Git as the single source of truth**.
- It continuously checks your cluster and makes sure the real state matches the desired state in Git.

3. The GitOps Model



4. What Problem Does It Solve?

- Automates application deployments
- Reduces manual work and human errors
- Ensures consistency across environments
- Detects and fixes configuration drift
- Provides audit history through Git
- Makes rollbacks simple and reliable

5. Key Benefits

- ✓ Automated and consistent deployments
- ✓ Uses Git as the source of truth
- ✓ Detects and corrects drift automatically
- ✓ Works with plain YAML, Helm and Kustomize
- ✓ Easy rollbacks
- ✓ Great for multiple environments

6. Real-World Example

A team stores their Kubernetes manifests in a Git repository. When a developer pushes a change, Argo CD automatically syncs the cluster to match the new desired state. No manual **kubectl apply** is needed.

7. Where Argo CD Fits

- Works with Kubernetes (any cloud or on-prem)
- Integrates with Git, Helm, Kustomize
- Used by DevOps, Platform and SRE teams
- Ideal for microservices and large environments
- Often part of a full GitOps toolchain



Instagram
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



Telegram
@veriqta



Medium
@veriqta



X
@veriqta

GitOps Before Argo CD

Git as source of truth, declarative configuration, reconciliation, drift

1. What is GitOps?

- GitOps is a way to manage infrastructure and applications using **Git as the single source of truth**.
- Everything is defined as code (**declarative configuration**).
- Changes are made through pull requests, reviewed and then applied to the cluster automatically.

2. Key Concepts



Git as Source of Truth

All desired configuration is stored in a Git repository (e.g. Kubernetes YAML, Helm charts, Kustomize).



Declarative Configuration

You define the desired state of your infrastructure and applications, not step-by-step commands.



Reconciliation

A tool continuously checks the cluster and makes sure the actual state matches what is in Git.



Drift

Drift happens when the live cluster state is different from the desired state in Git.

3. How GitOps Works (High Level)



4. What Problem Does It Solve?

- Eliminates manual deployment steps
- Reduces configuration drift
- Provides an audit history through Git
- Enforces review and approval (pull requests)
- Makes deployments consistent and repeatable
- Works across multiple environments

5. Common Sources of Drift

- Manual changes made directly in the cluster
- Hotfixes that are not committed to Git
- Configuration changes made outside the GitOps process
- Differences between environments
- Temporary troubleshooting changes that are not reverted

6. Key Takeaway

GitOps uses Git as the source of truth and continuously ensures the cluster matches the desired configuration. **Argo CD** is one of the popular tools that implements GitOps.

7. Remember This



In GitOps, you do not manually change resources in the cluster. You change the configuration in Git and let a tool reconcile the cluster automatically.



Instagram
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



Telegram
@veriqta



Medium
@veriqta



X
@veriqta

How Argo CD Works

API server, repository server, application controller, Redis, reconciliation flow

1. What Is Argo CD?

- **Argo CD** is a Kubernetes native **GitOps** tool.
- It continuously deploys and keeps your applications in sync with the **desired state in Git**.
- It runs inside your Kubernetes cluster.

2. Main Components



API Server

Handles UI, CLI and API requests.



Repository Server

Fetches and caches manifests from Git.



Application Controller

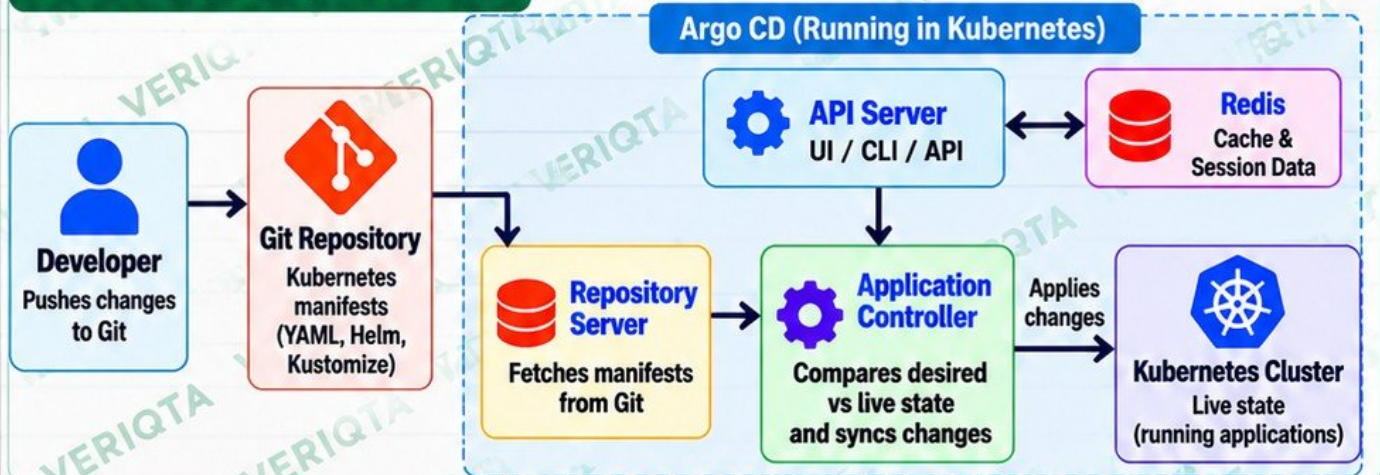
Compares desired state (Git) with live state and performs syncs.



Redis

Stores cache, session data and temporary state.

3. Architecture and Flow



4. Reconciliation Flow

- 1 Developer pushes changes to Git.
- 2 Repository server fetches the manifests from Git.
- 3 Application controller compares the desired state (Git) with the live state (cluster).
- 4 If there are differences, Argo CD syncs the changes to the cluster.
- 5 It continuously repeats this process (reconciliation loop).

5. Desired vs Live State

Desired State (in Git)



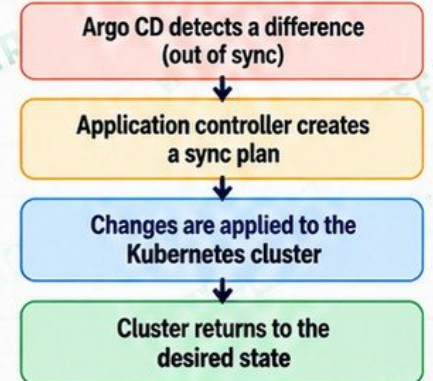
- Kubernetes manifests (YAML, Helm, Kustomize)
- Stored in a Git repository
- Represents the expected configuration

Live State (in Cluster)



- Actual running resources in Kubernetes
- May be different due to manual changes or failures
- Argo CD ensures both states match

6. What Happens if They Differ?



Instagram
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



Telegram
@veriqta



Medium
@veriqta



X
@veriqta

Installing Argo CD

Kubernetes prerequisites, namespace, installation, verifying components

1. Prerequisites

- ✓ A working Kubernetes cluster (v1.24 or later)
- ✓ kubectl configured and connected to the cluster
- ✓ Sufficient permissions (cluster-admin or required RBAC)
- ✓ Internet access to download Argo CD manifests

2. Create Namespace

Argo CD is typically installed in its own namespace called **argocd**.

```
$ kubectl create namespace argocd
```

namespace/argocd created



Kubernetes Namespace

argocd

3. Install Argo CD

Use the official manifest from the Argo CD project.

```
$ kubectl apply -n argocd \\  
-f https://raw.githubusercontent.com/  
argoproj/argo-cd/stable/manifests/install.yaml
```

resources created

! What This Does

- Deploys all Argo CD components in the **argocd** namespace
- Creates Deployments, Services, ConfigMaps and other resources
- Uses the official and recommended installation method

4. Verify Installation

Check that all Argo CD pods are running.

```
$ kubectl get pods -n argocd
```

NAME	READY	STATUS	RESTARTS	AGE
argocd-application-controller-0	1/1	Running	0	2m
argocd-dex-server-0	1/1	Running	0	2m
argocd-repo-server-0	1/1	Running	0	2m
argocd-server-0	1/1	Running	0	2m
argocd-redis-0	1/1	Running	0	2m

6. Check Services

Verify the services are created.

```
$ kubectl get svc -n argocd
```

NAME	TYPE	CLUSTER-IP	PORT(S)
argocd-metrics	ClusterIP	10.96.1.10	8082/TCP
argocd-server	ClusterIP	10.96.1.20	80/TCP,443/TCP
argocd-repo-server	ClusterIP	10.96.1.30	8081/TCP
argocd-redis	ClusterIP	10.96.1.40	6379/TCP

5. Argo CD Components

API Server (argocd-server)	→ Handles UI, CLI and API requests
Repository Server (argocd-repo-server)	→ Fetches and caches manifests from Git
Application Controller (argocd-application-controller)	→ Compares desired vs live state and performs syncs
Redis (argocd-redis)	→ Stores cache, session data and temporary state
Dex (argocd-dex-server)	→ Handles authentication and identity

7. Next Step

- Argo CD is now installed and running in your cluster.
- In the next page, you will learn how to access the Argo CD UI and CLI.
- Then you will deploy your first application.



Instagram
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



Telegram
@veriqta



Medium
@veriqta



X
@veriqta

Accessing Argo CD

Web UI, CLI, initial admin access, authentication basics, secure access

1. Access Methods

You can access Argo CD using:



Web UI

Browser-based interface for managing applications.



CLI

Command line tool for automation and scripting.



API

REST API used by the UI, CLI and integrations.

3. Accessing with CLI

Download the Argo CD CLI for your system.

Linux

macOS

Windows

```
$ curl -sSL -o argocd \
  https://github.com/argoproj/argo-cd/releases/latest/
  download/argocd-linux-amd64
$ chmod +x argocd
$ sudo mv argocd /usr/local/bin/
$ argocd version
```

4. Get Initial Admin Password

The default username is admin.

```
$ kubectl get secret argocd-initial-admin-secret \
  -n argocd -o jsonpath="{.data.password}" | base64 -d
```

This command returns the initial admin password.
Use this to log in to the Web UI or CLI.

6. Authentication Options



Default Admin

Built-in admin user (change or disable in production).



RBAC

Create users and roles with fine-grained permissions.



SSO

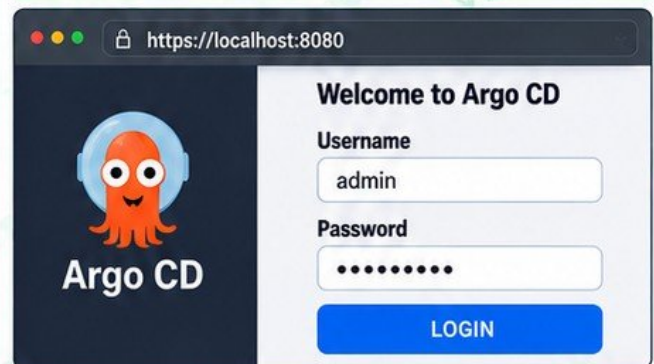
Integrate with identity providers (e.g. OIDC, LDAP, SAML).

2. Accessing the Web UI

Expose the Argo CD server and open it in your browser.

```
$ kubectl port-forward svc/argocd-server
  -n argocd 8080:443
```

Open your browser and go to:
<https://localhost:8080>



5. Login with CLI

Use the initial admin password to log in.

```
$ argocd login localhost:8080 \
  --username admin \
  --password <PASSWORD>
```

You should see a successful login message.

7. Secure Access (Production)

- ✓ Do not expose Argo CD publicly without authentication.
- ✓ Use RBAC and least privilege.
- ✓ Integrate with SSO (OIDC, LDAP or SAML).
- ✓ Use HTTPS with a valid certificate.
- ✓ Restrict access to trusted networks (VPN, bastion, etc).
- ✓ Disable the default admin user after setup.

8. Common Issues



- Cannot access UI: Check port-forward or Ingress.
- Invalid credentials: Ensure you used the correct password.
- CLI not found: Make sure the binary is in your PATH.
- TLS/SSL errors: Use a valid certificate or port-forward.



Instagram
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



Telegram
@veriqta



Medium
@veriqta



X
@veriqta

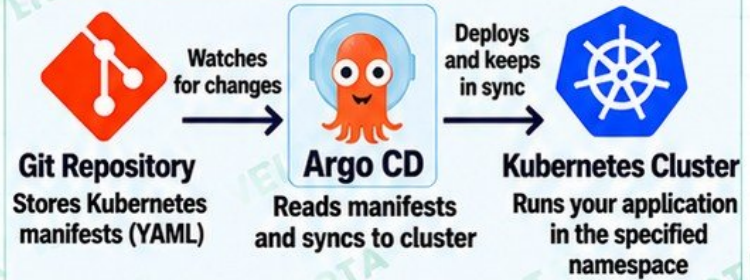
Your First Argo CD Application

Application object, Git repository, path, destination cluster, namespace

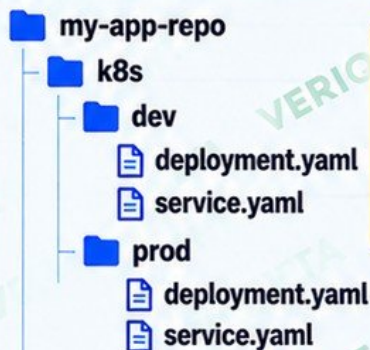
1. What Is an Argo CD Application?

- An Application tells Argo CD what to deploy, where to deploy it, and how to keep it in sync.
- It points to a **Git repository**, a **path** in the repository, a **destination cluster** and a **namespace**.
- Argo CD watches the repository and deploys the manifests to match the desired state.

2. High-Level Flow



3. Example Repository Structure



The path in the Application refers to the folder that contains your Kubernetes manifests.

4. Argo CD Application Manifest

```

apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: my-app
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/example/my-app-repo.git
    targetRevision: main
    path: k8s/dev
  destination:
    server: https://kubernetes.default.svc
    namespace: my-app
  syncPolicy:
    automated:
      prune: false
      selfHeal: false
  
```

This tells Argo CD:

- which repo to use
- which path to deploy
- which cluster to deploy to
- which namespace to use

5. Apply the Application

```
$ kubectl apply -f my-app.yaml
```

This creates the Application in Argo CD. Argo CD will then detect the repository, read the manifests and deploy the resources to your cluster.

6. Key Fields Explained

Field	What It Means
repoURL	URL of your Git repository.
targetRevision	Branch, tag or commit to track (e.g. main).
path	Path inside the repo where manifests are located.
destination.server	The Kubernetes cluster to deploy to.
destination.namespace	The namespace where resources will be created.

7. Verify in Argo CD

```
$ kubectl get applications -n argocd
```

You should see your application listed. You can also check it in the Argo CD Web UI.

8. Common Mistakes



- Wrong repository URL or path.
- Incorrect targetRevision (branch name).
- Deploying to the wrong cluster or namespace.
- Manifests not found in the specified path.
- Missing permissions to the Git repository.



Instagram
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



Telegram
@veriqta



Medium
@veriqta



X
@veriqta

Syncing an Application

OutOfSync vs Synced, manual sync, automated sync, sync status, health status

1. Sync Status vs Health Status



Sync Status

- Shows if the live state in the cluster matches the desired state in Git.
- Values: Synced, OutOfSync, Unknown.
- Indicates if Argo CD needs to take action (sync).



Health Status

- Shows if the application is running correctly in the cluster.
- Values: Healthy, Degraded, Progressing, Suspended, Unknown.
- Based on the actual state of the Kubernetes resources.

2. Sync and Health Status in the UI



argo

Applications

Settings

User Info

Documentation

Applications / my-app

my-app ☆

Sync Status

✓ Synced

Health Status

✓ Healthy

Repository <https://github.com/example/my-app.git>Target Revision [main](#)Path [k8s](#)Destination <https://kubernetes.default.svc>Namespace [my-app](#)

3. OutOfSync vs Synced



OutOfSync

- The live state in the cluster is different from what is in Git.
- Argo CD shows the differences.
- You need to sync the application (manually or automatically).



Synced

- The live state matches the desired state in Git.
- No changes are pending.
- The application is deployed as defined in the repository.

4. Manual Sync

You can manually sync an application from the UI or CLI.

REFRESH

SYNC

DELETE

Sync with latest revision

☐ Prune resources☐ Dry run☐ Force sync

CANCEL

SYNC

CLI

```
# Sync the application
argocd app sync my-app
```

```
# Check sync status
argocd app get my-app
```

5. Automated Sync

Argo CD can automatically sync when changes are pushed to Git.

```
spec:
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
  syncOptions:
    - CreateNamespace=true
```

6. Sync Options Explained

Option	Description
automated	Enables automatic sync when changes are detected in Git.
prune	Automatically deletes resources that are removed from Git.
selfHeal	Reverts manual changes made in the cluster to match Git.
CreateNamespace=true	Creates the destination namespace if it does not exist.

7. Check Application Status

Use CLI to check sync and health status.

```
# Get application details
argocd app get my-app
```

```
# Watch live status
argocd app get my-app --watch
```

8. Common Sync States

✓ Synced	No changes, in sync with Git.
✗ OutOfSync	Changes detected, needs sync.
❓ Unknown	Status cannot be determined.

9. Common Health States

 Healthy Running normally.	 Degraded Some resources are not healthy.	 Progressing Resources are still being created.	 Suspended Sync is paused.	 Unknown Health status cannot be determined.
----------------------------------	---	---	----------------------------------	--

10. Best Practices

- Always check both sync and health status.
- Use automated sync with care (prune and selfHeal).
- Monitor Argo CD notifications for failed syncs.
- Review changes before syncing in production.
- Use proper RBAC for who can sync applications.



Instagram
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



Telegram
@veriqta



Medium
@veriqta



X
@veriqta

Understanding Reconciliation and Drift

Desired vs live state, drift detection, self-healing, reconciliation loop

1. Desired vs Live State

Desired State (Git)



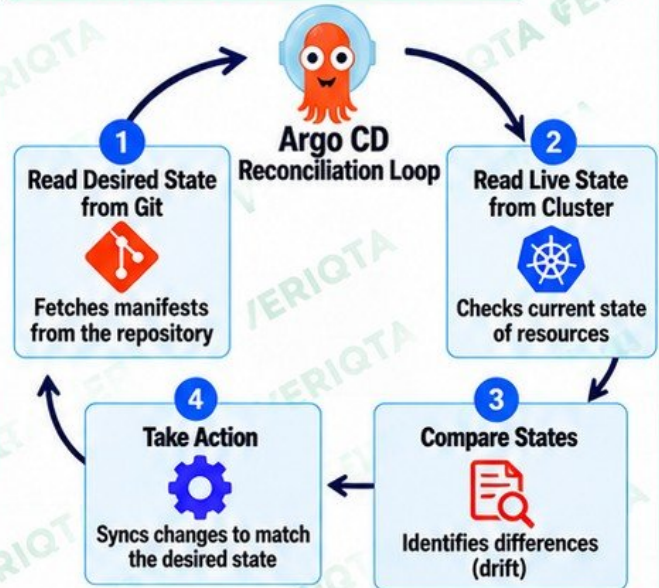
- Defined in Git repository as Kubernetes manifests (YAML, Helm, Kustomize).
- This is what you want the cluster to look like.
- Stored in version control.
- Example: 3 replicas, specific image version, environment variables.

Live State (Cluster)



- Actual state of resources running in the Kubernetes cluster.
- This is what is currently deployed.
- Can change due to manual changes, failed deployments or external systems.
- Example: 1 replica, different image, or modified configuration.

2. How Reconciliation Works



3. What Is Drift?



- Drift happens when the live state in the cluster is different from the desired state in Git.
- It can be caused by manual changes, hotfixes, failed deployments or other tools.
- Argo CD continuously detects drift and marks the application as **OutOfSync**.

4. Drift Example

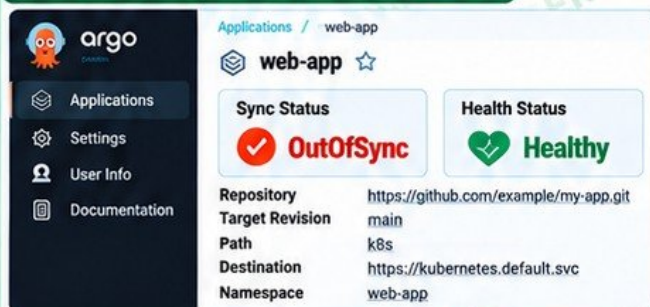
Desired State (Git)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
spec:
  replicas: 3
  template:
    spec:
      containers:
        - name: app
          image: nginx:1.25
```

Live State (Cluster)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
spec:
  replicas: 1 # drift
  template:
    spec:
      containers:
        - name: app
          image: nginx:1.24 # drift
```

5. Drift Detection in Argo CD



The screenshot shows the Argo CD web interface for an application named 'web-app'. The 'Sync Status' is 'OutOfSync' (indicated by a red checkmark icon) and the 'Health Status' is 'Healthy' (indicated by a green checkmark icon). The repository is 'https://github.com/example/my-app.git', target revision is 'main', path is 'k8s', destination is 'https://kubernetes.default.svc', and namespace is 'web-app'.

6. Self-Healing



- Self-healing automatically syncs the application when drift is detected.
- It reverts manual changes to match the desired state.
- Useful for maintaining consistency and preventing configuration drift.
- Enabled using: `automated.selfHeal: true`

7. Reconciliation Loop Behavior



- Argo CD continuously runs a reconciliation loop.
- It checks for changes at regular intervals (default ~3 minutes).
- If drift is detected, it marks the app **OutOfSync**.
- If automated sync is enabled, it will sync the changes automatically.

8. Key Takeaways



- Desired state is defined in Git.
- Live state is the actual state in the cluster.
- Drift is the difference between desired and live state.
- Argo CD detects drift and can automatically fix it.
- The reconciliation loop keeps your cluster in the desired state.



Instagram
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



Telegram
@veriqta



Medium
@veriqta



X
@veriqta

Automated Sync, Prune and Self-Heal

automated, prune, selfHeal, risks, safe automation practices

1. What These Options Do

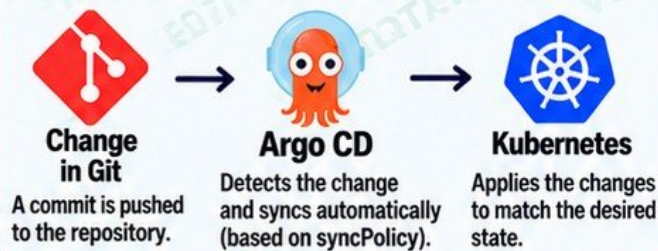
Option	What It Does
automated	Enables automatic sync when changes are detected in Git.
prune	Automatically deletes resources that are removed from Git.
selfHeal	Automatically reverts manual changes in the cluster to match the desired state in Git.

2. Example Application with Automated Sync

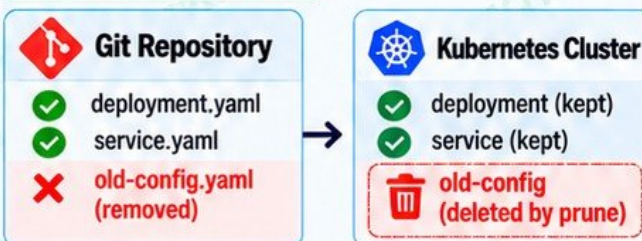
```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: web-app
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/example/web-app.git
    targetRevision: main
    path: k8s
  destination:
    server: https://kubernetes.default.svc
    namespace: web-app
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
```

This configuration enables automatic sync, prune and self-heal for the application.

3. How Automated Sync Works



4. Prune in Action



Prune removes resources from the cluster that no longer exist in the Git repository.

5. Self-Heal in Action



6. Risks and Considerations



Potential Risks

- Prune can delete important resources if misconfigured.
- Self-heal can overwrite manual changes (including emergency fixes).
- Incorrect Git repository or path can affect the wrong resources.
- Overly broad permissions can be dangerous.
- Automated sync can cause unintended changes in production if not reviewed.

7. Safe Automation Practices

- ✓ Use separate applications for different environments (dev, staging, production).
- ✓ Test automation in a non-production environment first.
- ✓ Use RBAC to restrict who can change Argo CD applications.
- ✓ Enable notifications (e.g. Slack, email) for sync events.
- ✓ Use application-level labels and projects for isolation.
- ✓ Review changes in Git before merging.
- ✓ Use sync windows if you need to restrict when changes can be applied.

8. Key Takeaways



- **automated** enables continuous delivery from Git.
- **prune** keeps the cluster clean by removing deleted resources.
- **selfHeal** prevents configuration drift.
- These features are powerful but should be used carefully.
- Always follow best practices to avoid accidental changes.



Instagram
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



Telegram
@veriqta



Medium
@veriqta



X
@veriqta

Managing Kubernetes Manifests

Plain YAML, directory structures, manifest discovery, repository organization

1. What Are Kubernetes Manifests?

- Kubernetes manifests are YAML files that define the desired state of your resources (e.g. Deployments, Services, Ingress).
- Argo CD reads these manifests from a Git repository and applies them to your cluster.
- Manifests tell Kubernetes what to create, up date or delete.
- You can manage plain YAML or use tools like Kustomize or Helm (covered later).

2. Example Plain YAML Manifest

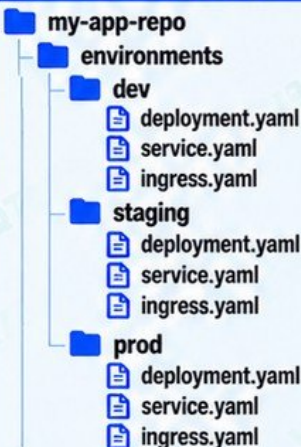
```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
  namespace: web-app
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
          ports:
            - containerPort: 80
```

deployment.yaml

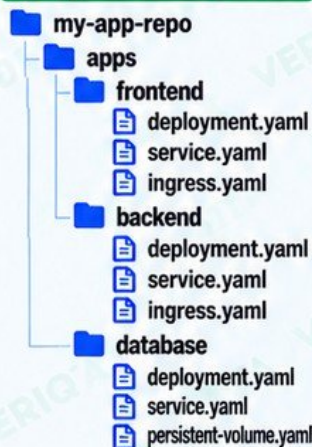
This YAML defines a Deployment with 3 replicas of nginx in the web-app namespace.

3. Common Directory Structures

Environment-Based



Application-Based



4. Manifest Discovery in Argo CD

- You specify the repository URL, revision (branch/tag) and the path where the manifests are located.
- Argo CD will automatically detect and apply all valid Kubernetes manifests in that path.
- It ignores non-manifest files (e.g. README.md).
- You can also use more advanced patterns like:
 - kustomization.yaml (Kustomize)
 - Chart.yaml (Helm)
 - ApplicationSet (for multiple environments)

5. Repository Organization Best Practices

- ✓ Use a clear and consistent directory structure.
- ✓ Separate environments (dev, staging, prod).
- ✓ Keep manifests simple and organized.
- ✓ Use meaningful names and labels.
- ✓ Store only Kubernetes manifests and related configuration.
- ✓ Include a README.md to explain the structure.
- ✓ Use Git branches or folders for different environments.
- ✓ Keep sensitive data out of the repository (use Secrets management).

6. Example Argo CD Application Configuration

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: my-app
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/example/my-app-repo.git
    targetRevision: main
    path: environments/dev
  destination:
    server: https://kubernetes.default.svc
    namespace: web-app
  syncPolicy:
    automated: selfHeal: false
```

application.yaml

Argo CD will look in the environments/dev folder and apply all Kubernetes manifests found there.

7. Common File Types Argo CD Can Use



8. Key Takeaways



- Manifests define the desired state of your cluster.
- Keep your repository organized and easy to maintain.
- Argo CD automatically discovers and applies manifests.
- Use clear directory structures and best practices.
- Keep sensitive information out of Git.

Helm with Argo CD

Helm charts, values, parameters, releases, GitOps deployment flow

1. What Is Helm?

- Helm is a package manager for Kubernetes.
- A Helm chart is a collection of Kubernetes manifests templated with values.
- It makes it easy to deploy, configure and manage applications.
- Argo CD can deploy Helm charts directly from a Git repository.



2. Key Helm Concepts

Concept	What It Means
Helm Chart	A package of Kubernetes manifests with templates.
values.yaml	Configuration file used to customize the chart.
Parameters	Values that override the defaults in values.yaml.
Release	A running instance of a Helm chart in Kubernetes.
Chart Repository	A place to store and version Helm charts (e.g. Git repo, Helm repo).

3. Helm Chart Structure

- my-app-chart/
 - Chart.yaml
 - values.yaml
 - templates/
 - deployment.yaml
 - service.yaml
 - ingress.yaml
 - charts/
 - (dependencies)

Chart.yaml contains metadata about the chart.
values.yaml contains default configuration.
The templates/ folder contains Kubernetes manifests templated with values.

4. Example values.yaml

```
replicaCount: 2
image:
  repository: nginx
  tag: 1.25
  pullPolicy: IfNotPresent
service:
  type: ClusterIP
  port: 80
ingress:
  enabled: true
  host: myapp.example.com
  tls: false
resources:
  limits:
    cpu: 500m
    memory: 512Mi
  requests:
    cpu: 250m
    memory: 256Mi
```

values.yaml

You can override these values using a custom values.yaml file or individual parameters in Argo CD.

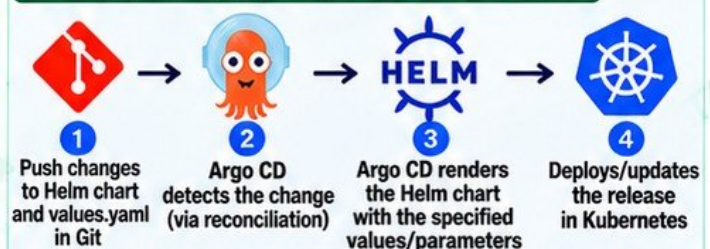
5. Argo CD Helm Configuration

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: my-helm-app
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/example/my-helm-repo.git
    targetRevision: main
    path: charts/my-app
    helm:
      valueFiles:
        - values.yaml
      parameters:
        - name: image.tag
          value: "1.25"
  destination:
    server: https://kubernetes.default.svc
    namespace: my-app
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

application.yaml

Argo CD uses the helm section to specify value files and parameters for the chart.

6. GitOps Deployment Flow with Helm



7. Parameters vs values.yaml

Method	When to Use
values.yaml	For environment-specific configuration files (e.g. dev, staging, prod).
Parameters	To override specific values without a separate file.
Both	You can use both value files and parameters together.

8. Best Practices

- Keep your Helm charts and values.yaml in a Git repository.
- Use separate values files for each environment (dev, staging, prod).
- Use parameters for small overrides (e.g. image tag).
- Enable automated sync with prune and selfHeal (when appropriate).
- Keep charts simple and well documented.

9. Common Mistakes



- Using the wrong path to the chart.
- Missing or incorrect values.yaml.
- Overriding values incorrectly.
- Not versioning your Helm charts.
- Giving Argo CD insufficient permissions.
- Using one values file for all environments.

Kustomize with Argo CD

Bases, overlays, environment differences, production configuration

1. What Is Kustomize?

- Kustomize is a built-in Kubernetes tool for customizing YAML manifests.
- You define a base configuration and overlays for different environments (dev, staging, prod).
- It lets you manage environment differences without using templates.
- Argo CD can deploy Kustomize directly from your Git repository.



3. Base Example (base/kustomization.yaml)

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
- deployment.yaml
- service.yaml
- configmap.yaml
```

The base defines the common resources for your application.

4. Overlay Example (overlays/prod/kustomization.yaml)

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namePrefix: prod-
resources:
- ../../base
patches:
- path: patch-replicas.yaml
- path: patch-resources.yaml
images:
- name: my-app
  newTag: 2.0.0
```

The prod overlay uses the base and applies changes like replicas, resource limits and image tags.

5. Example Patch (overlays/prod/patch-replicas.yaml)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 5
```

This patch changes the number of replicas for the production environment.

8. Best Practices

- ✓ Keep a clean base and use overlays for environment-specific changes.
- ✓ Only put the differences in each overlay.
- ✓ Use meaningful names for directories and patches.
- ✓ Version your configuration in Git.
- ✓ Review changes before deploying to production.
- ✓ Use automated sync with care in production.

2. Kustomize Structure (Base and Overlays)

```
my-app/
├── base/
│   ├── kustomization.yaml
│   ├── deployment.yaml
│   ├── service.yaml
│   └── configmap.yaml
└── overlays/
    ├── dev/
    │   ├── kustomization.yaml
    │   ├── patch-replicas.yaml
    │   └── patch-resources.yaml
    ├── staging/
    │   ├── kustomization.yaml
    │   ├── patch-replicas.yaml
    │   └── patch-resources.yaml
    └── prod/
        ├── kustomization.yaml
        ├── patch-replicas.yaml
        └── patch-resources.yaml
```

base/

Contains the common manifests used in all environments.

overlays/

Contains environment-specific customizations using patches.

Each overlay only contains the changes for that environment (e.g. replicas, resources, image tags, config).

6. Deploying with Argo CD

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: my-app
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/example/my-app.git
    targetRevision: main
    path: overlays/prod # or dev, staging
    kustomize: {}
  destination:
    server: https://kubernetes.default.svc
    namespace: my-app
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

Argo CD will deploy the Kustomize configuration from the specified path (e.g. overlays/prod).

7. Environment Differences

Environment	Replicas	Image Tag	Resources
dev	1	latest	Low (CPU/Memory)
staging	2	1.0.0	Medium
prod	5	2.0.0	Higher (with limits)

9. Key Takeaways



- Kustomize makes it easy to manage environment differences.
- Argo CD can deploy Kustomize natively from Git.
- Use overlays for dev, staging and production.
- Keep your configuration simple and well organized.
- Always review production changes before syncing.



Instagram
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



Telegram
@veriqta



Medium
@veriqta



X
@veriqta

Managing Multiple Environments

Dev, staging, production, repository patterns, promotion strategies

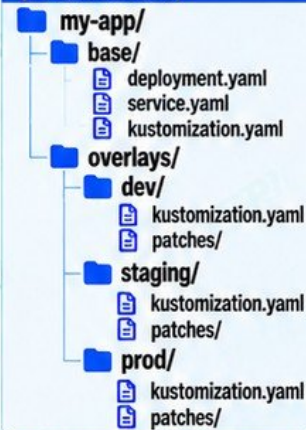
1. Why Multiple Environments?

- Separate environments reduce risk.
- Each environment can have different configuration, resources and limits.
- You can test changes in dev and staging before promoting to production.
- Argo CD makes it easy to manage multiple environments from Git.

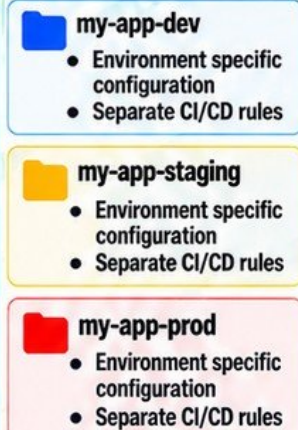


2. Common Repository Patterns

Pattern 1: Separate Folders (Recommended)



Pattern 2: Separate Repositories



3. Argo CD Applications for Each Environment

Dev Application

```

apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: my-app-dev
  namespace: argocd
spec:
  source:
    repoURL: https://github.com/example/my-app.git
    path: overlays/dev
    targetRevision: main
  destination:
    server: https://kubernetes.default.svc
    namespace: my-app-dev
  
```

Staging Application

```

apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: my-app-staging
  namespace: argocd
spec:
  source:
    repoURL: https://github.com/example/my-app.git
    path: overlays/staging
    targetRevision: main
  destination:
    server: https://kubernetes.default.svc
    namespace: my-app-staging
  
```

Production Application

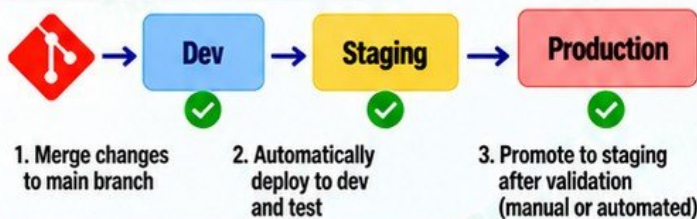
```

apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: my-app-prod
  namespace: argocd
spec:
  source:
    repoURL: https://github.com/example/my-app.git
    path: overlays/prod
    targetRevision: main
  destination:
    server: https://kubernetes.default.svc
    namespace: my-app-prod
  
```

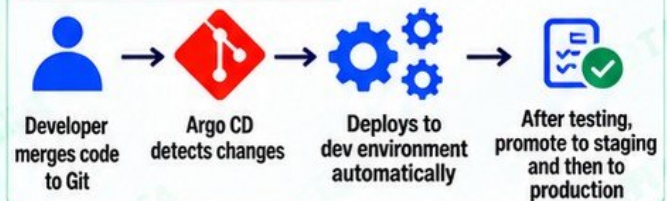
4. Environment Differences

Setting	Dev	Staging	Production
Replicas	1	2	5
CPU Limit	250m	500m	1
Memory Limit	512Mi	1Gi	2Gi
Image Tag	latest	1.0.0	1.0.0
Ingress	dev.example.com	staging.example.com	app.example.com
Logging Level	debug	info	warn
Auto Sync	Yes	Yes	Yes (with care)

5. Promotion Strategies



6. Example Workflow



7. Best Practices

- ✓ Use a consistent directory structure.
- ✓ Keep environment-specific configuration in overlays or separate repos.
- ✓ Use clear naming for applications and namespaces.
- ✓ Set resource limits appropriate for each environment.
- ✓ Use manual approval for production if required.
- ✓ Monitor each environment and set up alerts.
- ✓ Keep your promotion process simple and well documented.

8. Common Mistakes



- Using the same configuration for all environments.
- Deploying directly to production without testing.
- Not setting resource limits for production.
- Hardcoding environment-specific values.
- Forgetting to review changes before promoting.
- Using automatic sync for production without proper controls.

9. Key Takeaways

- ✓ Use Git to manage multiple environments.
- ✓ Keep environment differences clear and organized.
- ✓ Test in dev and staging before promoting to production.
- ✓ Automate where possible but use controls for production.



Instagram
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



Telegram
@veriqta



Medium
@veriqta



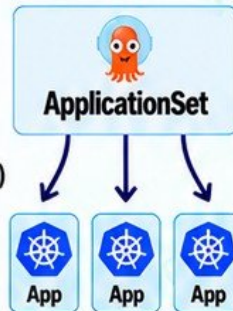
X
@veriqta

ApplicationSet

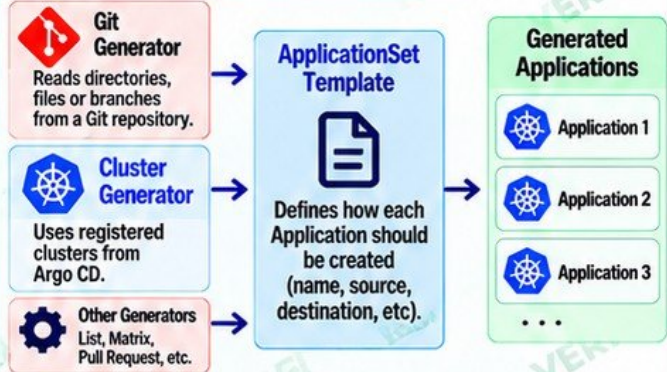
Generating applications, Git generator, cluster generator, scaling deployments

1. What Is ApplicationSet?

- ApplicationSet is an Argo CD resource that automatically generates multiple Applications based on templates.
- It helps you manage many applications at scale.
- You define generators (Git, cluster, etc.) that provide input data.
- Argo CD creates and manages the Applications for you.



2. How ApplicationSet Works



3. Example: Git Generator

```
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: git-apps
spec:
  generators:
    - git:
        repoURL: https://github.com/example/apps.git
        revision: main
        directories:
          - path: apps/*
  template:
    metadata:
      name: '{{path.basename}}'
    spec:
      project: default
      source:
        repoURL: https://github.com/example/apps.git
        targetRevision: main
        path: '{{path}}'
      destination:
        server: https://kubernetes.default.svc
        namespace: '{{path.basename}}'
```

applicationset.yaml

This generator creates an Application for each directory inside apps/* in the repository.

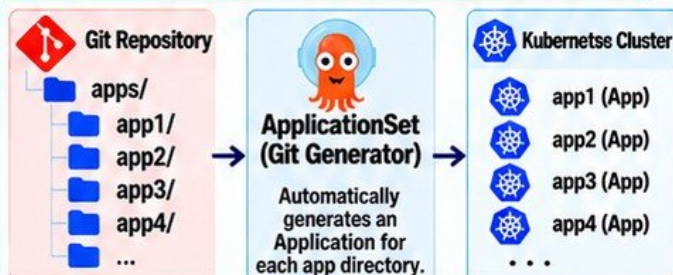
4. Example: Cluster Generator

```
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: cluster-apps
spec:
  generators:
    - clusters:
        selector:
          matchLabels:
            environment: prod
  template:
    metadata:
      name: 'my-app-{{name}}'
    spec:
      project: default
      source:
        repoURL: https://github.com/example/my-app.git
        targetRevision: main
        path: app
      destination:
        server: '{{server}}'
        namespace: my-app
```

applicationset.yaml

This generator creates an Application for every cluster that matches the label environment=prod.

5. Scaling Deployments with ApplicationSet



7. Best Practices

- ✓ Use clear and consistent naming for generated applications. Organize your repository structure for easy scaling.
- ✓ Use labels and selectors for cluster generator.
- ✓ Set resource limits and sync policies appropriately.
- ✓ Test ApplicationSet in non-production first.
- ✓ Monitor generated applications and set up alerts.

6. Common Generators

Generator	Use Case	Example Input
Git Generator	Create apps from Git directories, files or branches.	app1/, app2/
Cluster Generator	Create apps for each cluster.	All clusters or label selectors
List Generator	Create apps from a static list of values.	List of environments
Matrix Generator	Combine multiple generators (e.g. Git + clusters).	Git directories x clusters
Pull Request Generator	Create apps for each PR (e.g. preview environments).	Open pull requests

8. Common Mistakes

- ! Incorrect path patterns in Git generator.
- ! Generating too many applications accidentally.
- ! Missing labels on clusters for cluster generator.
- ! Not using meaningful names or namespaces.
- ! Overusing ApplicationSet for simple use cases.

9. Key Takeaways

- ✓ ApplicationSet helps you manage many applications at scale.
- ✓ Use Git or cluster generators based on your use case.
- ✓ It is powerful for multi-environment and multi-cluster deployments.
- ✓ Keep templates simple, organized and maintainable.

Sync Waves, Hooks and Deployment Order

PreSync, Sync, PostSync, sync waves, dependencies, controlled rollout

1. What Are Sync Waves and Hooks?

- Sync waves control the order in which resources are applied.
- Hooks run custom jobs or tasks at different stages of the sync process.
- This helps manage dependencies and gives you a controlled rollout.
- Common hook types: PreSync, Sync, PostSync, SyncFail, PostDelete.
- Use sync waves and hooks to ensure critical components are deployed in the right order.



2. Sync Lifecycle and Hook Types



Common Use Cases:

- **PreSync:** run database migrations, create namespaces, run checks.
- **Sync:** deploy your application resources.
- **PostSync:** run smoke tests, send notifications.
- **SyncFail:** collect logs, alert on failure.
- **PostDelete:** cleanup resources.

3. Sync Waves for Deployment Order

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  annotations:
    argocd.argoproj.io/sync-wave: "1"
spec:
  replicas: 3
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: backend
          image: myapp/backend:1.0.0
  
```

deployment.yaml

Resources with a lower sync-wave number are applied first. In this example, backend will be deployed in wave 1.

4. Hook Example (PreSync Job)

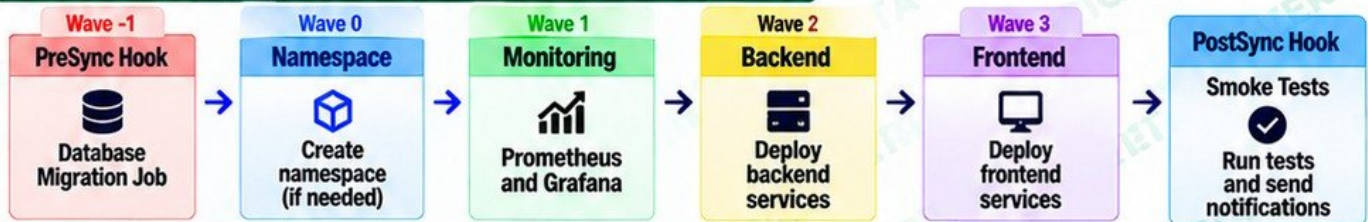
```

apiVersion: batch/v1
kind: Job
metadata:
  name: db-migration
  annotations:
    argocd.argoproj.io/hook: PreSync
    argocd.argoproj.io/hook-delete-policy:
      - HookSucceeded
spec:
  template:
    spec:
      containers:
        - name: migrate
          image: myapp/migrate:1.0.0
          command: ["/bin/sh", "-c"]
          args: ["/migrate.sh"]
          restartPolicy: Never
  
```

pre-sync-job.yaml

This job runs before the main application is synced. It can be used for database migrations or pre-deployment checks.

5. Example Deployment Order with Waves and Hooks



6. Controlling Rollouts and Dependencies

- ✓ Use sync waves to define the order of dependent resources.
- ✓ Use hooks to run custom tasks before and after deployment.
- ✓ Ensure critical components (e.g. databases, CRDs) are deployed first.
- ✓ Use health checks and PostSync tests for safer rollouts.
- ✓ Keep your manifests organized with clear annotations.

7. Common Mistakes

- ! Deploying dependent resources without sync waves.
- ! Running database migrations after the application.
- ! Forgetting to clean up hook resources.
- ! Using the same sync wave for everything.
- ! Not handling sync failures with SyncFail hooks.

8. Key Takeaways

- ✓ Sync waves and hooks give you control over deployment order.
- ✓ Use PreSync, Sync and PostSync for reliable deployments.
- ✓ Always consider dependencies between resources.
- ✓ Test your rollout process in a non-production environment first.

Secrets and Security

Why plaintext secrets are dangerous, secret-management patterns, RBAC, least privilege

1. Why Plaintext Secrets Are Dangerous?

- Plaintext secrets can be read by anyone with access to the Git repository.
- Secrets in Git history are hard to remove completely.
- They can be exposed through logs, backups, CI/CD systems and developer machines.
- Attackers can gain access to your clusters, databases and cloud accounts.
- Always use a proper secret management solution instead of storing real secrets in Git.



Never commit real secrets to Git.

3. Example: Kubernetes Secret (NOT for real secrets)

```
apiVersion: v1
kind: Secret
metadata:
  name: my-app-secret
  namespace: my-app
type: Opaque
data:
  DB_USER: bXlhchA=
  DB_PASSWORD: c2VjdXJlUGFzc3dvcmQ=
```

secret.yaml

This is base64 encoded, not encrypted. Do not store real secrets like this in Git.

2. Secret Management Patterns

Pattern	Description	Use Case
Kubernetes Secrets	Store secrets in the cluster (encoded, not encrypted by default).	Simple use cases, non-sensitive data.
External Secrets Operator	Sync secrets from external systems (AWS Secrets Manager, HashiCorp Vault, Azure Key Vault).	Production environments.
Sealed Secrets	Encrypt secrets so they can be safely stored in Git.	GitOps with encryption.
SOPS (Age/GPG)	Encrypt secret files in Git using Age or GPG.	GitOps with encrypted YAML.
Vault Integration	Use HashiCorp Vault to manage and inject secrets dynamically.	Enterprise secret management.

4. Example: External Secrets Operator

```
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: my-app-secret
  namespace: my-app
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: aws-secrets-manager
    kind: ClusterSecretStore
  target:
    name: my-app-secret
    creationPolicy: Owner
  data:
    - secretKey: DB_PASSWORD
      remoteRef:
        key: prod/my-app/db-password
```

externalsecret.yaml

External Secrets Operator pulls secrets from an external store like AWS Secrets Manager and creates a Kubernetes Secret automatically.

5. Argo CD and Secrets Management



6. RBAC for Argo CD (Example)

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: argocd-rbac-cm
  namespace: argocd
data:
  policy.csv: |
    p, role:readonly, applications, get, *, allow
    p, role:dev, applications, *, my-app-*, allow
    p, role:ops, applications, *, *, allow
  policy.default: role:readonly
```

argocd-rbac-cm.yaml

Use RBAC to control who can access Argo CD and what they can do. Give only the permissions that are required (least privilege).

7. Least Privilege Principles

- ✓ Give users and services only the permissions they need.
- ✓ Use separate roles for different teams (dev, staging, prod).
- ✓ Restrict access to sensitive resources and namespaces.
- ✓ Use read-only access for viewing applications.
- ✓ Audit access regularly and remove unused permissions.
- ✓ Use short-lived credentials where possible.

8. Common Mistakes



- Storing plaintext secrets in Git.
- Using base64 secrets thinking they are encrypted.
- Giving admin access to everyone.
- Not using RBAC or using overly permissive roles.
- Exposing secrets in logs or CI/CD systems.
- Not rotating secrets regularly.

9. Key Takeaways

- ✓ Never store real secrets in plaintext in Git.
- ✓ Use a secret management solution (Vault, External Secrets, Sealed Secrets, etc).
- ✓ Implement RBAC and follow least privilege.
- ✓ Keep your secrets secure, rotated and well managed.

Production Deployment Strategies

Rolling updates, canary and blue-green concepts, Argo Rollouts relationship

1. Why Deployment Strategy Matters?

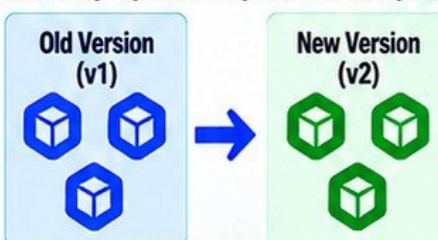
- Reduces downtime and risk.
- Helps you release new versions safely.
- Allows gradual traffic shift and quick rollback.
- Protects your users and revenue.
- Helps you detect issues before full rollout.



A good deployment strategy is not just about deploying. It's about controlling risk.

2. Rolling Update (Default Kubernetes)

Gradually replaces old pods with new pods.



Key Points

- Updates a few pods at a time.
- No downtime.
- Simple and widely used.
- Good for most applications.

3. Rolling Update Example (Deployment)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1 # create 1 extra pod
      maxUnavailable: 1 # allow 1 pod down
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: app
          image: myapp:2.0.0
```

deployment-rolling.yaml

Rolling update replaces pods gradually using maxSurge and maxUnavailable.

4. Canary Deployment

Release to a small percentage of users first.

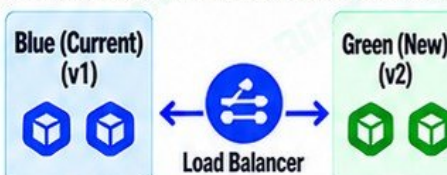


Key Points

- Send a small % of traffic to new version.
- Monitor metrics and errors.
- Gradually increase traffic.
- Reduce risk of major issues.

5. Blue-Green Deployment

Run two identical environments and switch traffic.



Key Points

- Both versions run at the same time.
- Switch traffic when ready.
- Instant rollback if needed.
- Requires more resources.

6. Argo Rollouts and Advanced Strategies



Argo Rollouts

- Argo Rollouts is a Kubernetes controller for advanced deployments.
- Supports rolling update, canary, blue-green and traffic management.
- Works with Argo CD (GitOps).
- Provides analysis, automated rollback and progressive delivery.

Key Features

- Canary and blue-green deployments.
- Traffic shifting (with Ingress, service mesh).
- Automated analysis and rollback.
- Works alongside Argo CD.

7. Canary Example with Argo Rollouts

```
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: my-app
spec:
  replicas: 3
  strategy:
    canary:
      steps:
        - setWeight: 10
        - pause: { duration: 5m }
        - setWeight: 50
        - pause: { duration: 5m }
        - setWeight: 100
```

rollout-canary.yaml

This example releases 10% traffic first, waits 5 minutes, then 50%, then 100% using Argo Rollouts.

8. Comparison of Deployment Strategies

Strategy	How It Works	Key Benefits	Best Use Case
Rolling Update	Gradually replaces pods	Simple, no downtime	Most applications
Canary	Small % of traffic first	Lower risk, early feedback	Critical applications
Blue-Green	Switch between environments	Instant rollback	High availability apps
Argo Rollouts	Advanced traffic control	Canary, blue-green, analysis	Production and large scale

9. Common Mistakes



- Deploying directly to all users without testing.
- Not monitoring after deployment.
- Ignoring rollback strategy.
- Not setting resource limits and health checks.
- Using blue-green without enough resources.

10. Key Takeaways



- Choose the right strategy based on application risk and impact.
- Use canary or blue-green for critical services.
- Use Argo Rollouts for advanced deployment control.
- Always monitor metrics and have a rollback plan.
- Test your deployment strategy in a non-production environment first.

Troubleshooting Argo CD

OutOfSync, Degraded, Unknown, failed syncs, repository errors, controller logs

1. Argo CD Application Health Status



Healthy

All resources are running as expected.



OutOfSync

The live state in the cluster is different from the desired state in Git.



Degraded

One or more resources are not healthy or have errors.



Unknown

Argo CD cannot determine the health status of the resources.

2. Common Issues and Causes

Issue	What It Means	Common Causes
 OutOfSync	Live state differs from Git.	- Manual changes in cluster - Drift in configuration - Image tag updated outside Git
 Degraded	Resources are not healthy.	- Pod crashes or not ready - Invalid configuration - Missing dependencies - Failed hooks
 Unknown	Health status cannot be determined.	- Custom resources without health checks - CRDs not installed - Argo CD cannot access resource status
 Sync Failed	Application sync was not successful.	- Repository errors - Invalid manifests - Permissions issues - Failed hooks or validation errors

3. Check Application Status

Check all applications

```
argocd app list
```

Get detailed app status

```
argocd app get my-app
```

Check app sync and health

```
argocd app get my-app -o wide
```

Look at the SYNC STATUS and HEALTH STATUS to quickly identify issues.

4. Common Troubleshooting Commands

Describe application (events, errors, conditions)

```
argocd app describe my-app
```

View application resources

```
argocd app resources my-app
```

View application logs (controller)

```
kubectl logs -n argocd deployment/argocd-application-controller
```

Check recent events

```
kubectl get events -n argocd --sort-by=.lastTimestamp
```

View repository connection status

```
argocd repo list
```

Manually trigger a sync

```
argocd app sync my-app
```

5. Repository Errors



Common Causes

- Wrong repository URL
- Invalid credentials (SSH key / token)
- Network or DNS issues
- Repository not accessible
- Branch, path or revision not found

Check Repository

```
argocd repo list
```

```
argocd repo get <repo-name>
```

6. Debug Failed Syncs

Check sync history

```
argocd app history my-app
```

View the last sync operation

```
argocd app get my-app -o json | jq '.status.operationState'
```

Check sync operation logs

```
kubectl logs -n argocd deployment/argocd-application-controller | tail -n 100 | grep my-app
```

Look for validation errors, manifest errors, hook failures or permission issues in the logs.

8. Common Fixes

- Revert manual changes and sync from Git.
- Fix invalid manifests or configuration errors.
- Ensure repository access and credentials are correct.
- Check that all required CRDs and dependencies are installed.
- Verify RBAC permissions for Argo CD.
- Re-run the sync after fixing the issue.
- Monitor controller logs for any remaining errors.

7. Controller Logs and Debugging

Application controller logs

```
kubectl logs -n argocd deployment/argocd-application-controller
```

Repo server logs (Git operations)

```
kubectl logs -n argocd deployment/argocd-repo-server
```

Server logs (API/UI)

```
kubectl logs -n argocd deployment/argocd-server
```

Follow logs in real time

```
kubectl logs -n argocd deployment/argocd-application-controller -f
```

9. Best Practices

- Keep Git as the single source of truth.
- Avoid making manual changes in the cluster.
- Use clear error messages and monitoring/alerts.
- Validate manifests before committing to Git.
- Keep repository credentials secure.
- Set up notifications for sync failures.
- Regularly check controller logs and application health.

10. Key Takeaways

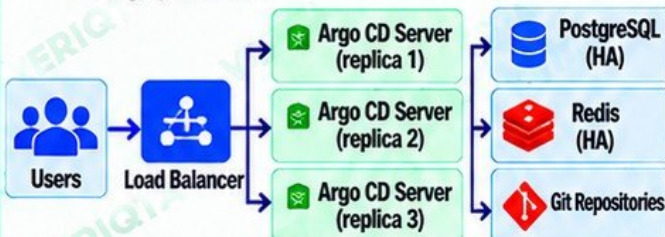
- Use application status and logs to identify the root cause.
- Most issues are configuration, repository or permission related.
- Fix the error, then sync again.
- Keep your Git repository clean and accessible.

Operating Argo CD in Production

High availability, RBAC, SSO, observability, backups, upgrades, disaster recovery

1. High Availability (HA) Deployment

- Run multiple replicas of Argo CD components for high availability.
- Use a highly available Kubernetes cluster and persistent storage (e.g. PostgreSQL).
- Place the Argo CD Server behind a load balancer.
- Ensure etcd, database and repository access are highly available.



2. RBAC and Access Control

- Use RBAC to control who can access Argo CD and what they can do.
- Define roles and policies for different teams (e.g. read-only, application admin, cluster admin).
- Use projects to restrict access to specific clusters, namespaces and repositories.

```
apiVersion: v1
kind: ConfigMap
```

```
metadata:
  name: argocd-rbac-cm
```

```
data:
  policy.csv: |
    p, role:readonly, applications, get, /*, allow
    p, role:dev, applications, sync, my-project/*, allow
    p, role:admin, *, *, allow
  policy.default: role:readonly
```

argocd-rbac-cm.yaml

Define fine-grained permissions using RBAC and projects.

3. SSO (Single Sign-On)

- Integrate Argo CD with your identity provider.
- Common options: OIDC (e.g. Azure AD, Okta, GitHub).
- Enforce SSO for secure, centralized access management.

```
data:
  dex.config: |
    connectors:
      - type: oidc
        id: azure
        name: Azure AD
        config:
          issuer: https://login.microsoftonline.com/<tenant-id>
          clientId: <client-id>
          clientSecret: <client-secret>
          scopes: ["openid", "profile", "email"]
```

argocd-cm.yaml

Use OIDC/SAML for SSO and enforce MFA through your identity provider.

4. Observability and Monitoring

- Monitor Argo CD components and application health.
- Use Prometheus (metrics) and Grafana (dashboards).
- Set up alerts for sync failures, unhealthy applications and controller errors.



5. Backups and Disaster Recovery

- Backup Argo CD database (PostgreSQL) regularly.
- Backup critical configuration (argocd-cm, argocd-rbac-cm, projects).
- Store backups in a separate location (e.g. S3, object storage).
- Test restore process periodically.



6. Upgrades and Maintenance

- Follow official upgrade guides for Argo CD.
- Test upgrades in a non-production environment first.
- Use a controlled rollout strategy.
- Check for breaking changes in release notes.



8. Common Mistakes

- Running Argo CD without HA in production.
- Not configuring proper RBAC and access controls.
- Skipping backups of the database and configuration.
- Upgrading without testing in a non-production environment.
- Not monitoring Argo CD or setting up alerts.

7. Disaster Recovery Strategy

- Document your recovery procedures.
- Keep infrastructure, manifests and secrets backed up.
- Be prepared for cluster or region failure.
- Regularly run disaster recovery drills.



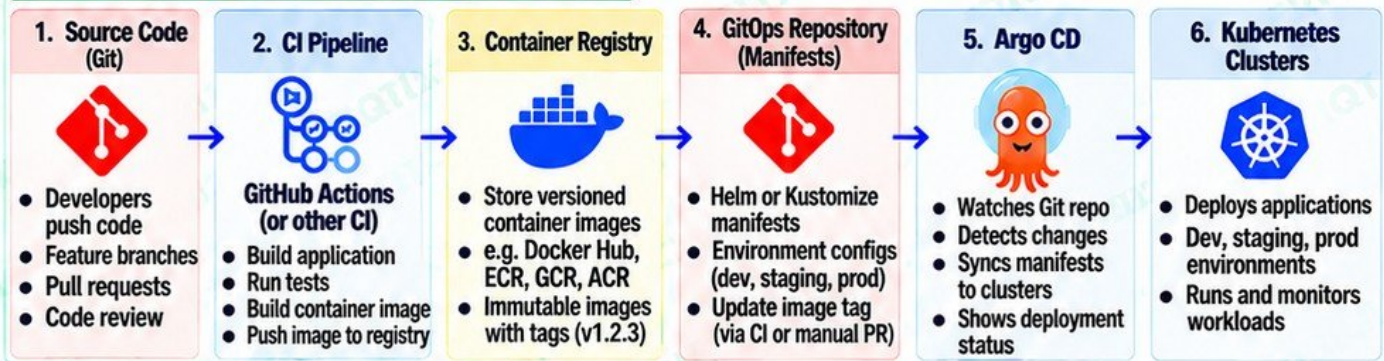
9. Key Takeaways

- Run Argo CD with high availability and persistent storage.
- Use SSO and RBAC for secure access.
- Monitor, set up alerts and keep backups.
- Follow a proper upgrade and disaster recovery plan.
- Keep your Argo CD environment secure, observable and well maintained.

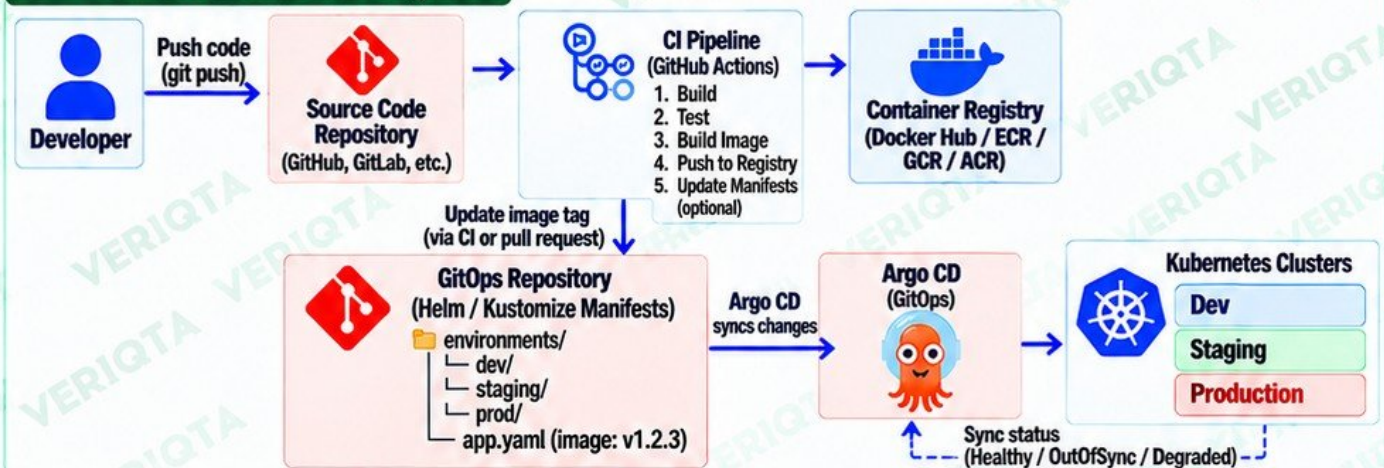
Production GitOps Architecture

End-to-end Git → CI → registry → GitOps repo → Argo CD → Kubernetes workflow

1. End-to-End Production GitOps Workflow



2. Detailed Architecture Diagram



3. Example CI Pipeline (GitHub Actions)

```
name: Build and Push Image
on:
  push:
    branches: [ main ]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build Docker Image
        run: docker build -t myapp:${{ github.sha }}
      - name: Push to Registry
        run: |
          docker login ${{ secrets.REGISTRY }} -u ${{ secrets.USER }} -p ${{ secrets.PASS }}
          docker push myapp:${{ github.sha }}
      - name: Update GitOps Repo (optional)
        run: |
          echo "Update image tag in GitOps repo"
          # e.g. update Helm values or Kustomize image tag
```

.github/workflows/ci.yaml

Build, test, push an image and optionally update the GitOps repository with the new image tag.

4. Argo CD Application (Example)

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: my-app
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com
    targetRevision: main
    path: environments/prod
  destination:
    server: https://kubernetes.default.svc
    namespace: my-app
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

application.yaml

Argo CD watches this repository and deploys the manifests to the Kubernetes cluster automatically.

5. Production Checklist

- ✓ Source code stored in Git with branch protection.
- ✓ CI pipeline builds, tests and pushes images.
- ✓ Images stored in a secure container registry.
- ✓ Manifests/Helm charts stored in a GitOps repo.
- ✓ Image tags are updated automatically or via PR.
- ✓ Argo CD syncs and deploys to the correct cluster.
- ✓ Environment separation (dev, staging, prod).
- ✓ Proper RBAC and access control.
- ✓ Secrets managed securely (no plaintext).
- ✓ Monitoring and alerting in place.
- ✓ Backups and disaster recovery tested.
- ✓ Rollback process verified.

6. Common Pitfalls

- ! Deploying from the wrong branch.
- ! Using latest tag instead of versioned tags.
- ! No environment separation.
- ! Storing secrets in Git.
- ! Overly permissive RBAC.
- ! No monitoring or alerting.
- ! Not testing the rollback process.
- ! Manual changes in the cluster.